Fuzz Testing for Input Validation

Pizza Order Application | 11 November 2026

Result: Six generated robustness tests fail on the original starter. After validation fixes, all **31 tests pass**. The corrected CLI handles invalid input without an uncaught exception.

Application and provenance

The pizza order application continues the Lab 6 problem statement. This submission uses the assigned repository's Lab 7 input-processing starter, which contains toppings_price, pizza_inches, and split_bill. The starter pizza_order.py is renamed processor.py to match the submission instructions. The unchanged starter is preserved in original/processor.py.

Assigned source: SETAPESU26 / L6_L7_24_pizza_order / Lab 7
Source commit: e3fec37127d66b4195c35458bb8725b3f08f588f
Individual repository: ArundhathiK29 / SE-Labs-PES1UG24CS082 / Lab7

Test method and validation policy

Hypothesis generates Unicode text, None, numbers, booleans, bytes, lists, and dictionaries. Structured strategies check valid topping sums, normalized sizes, bill division, and zero divisors. Ten generated properties use up to 200 examples each; fixed regression cases cover malformed input and size limits. Generation is deterministic and the example database is disabled.

Invalid input returns **None**; the CLI prints a validation message. Known toppings and sizes ignore case and surrounding whitespace. Bills accept two unsigned ASCII integer fields of 1-12 digits, separated by one slash; people must be greater than zero. Inputs are limited to 4096 characters. Valid results preserve the starter's pricing, dimensions, and rounding.

Execution and evidence

Environment: Windows, Python 3.11.9, pytest 9.1.1, Hypothesis 6.170.15. Screenshots below are browser captures of saved, unedited subprocess logs. Input lines, commands, timestamps in IST, and exit codes are shown above each capture. Full logs and their SHA-256 hashes accompany the submission. Fuzz success supports the tested properties; it does not prove every possible input.

Original application execution

Valid inputs succeed; a zero divisor terminates the original program.

01 original valid

Arundhathi K | PES1UG24CS082 | Lab 7

Command: python original/processor.py
Input lines: cheese,olives | M | 900/3
Exit code: 0 | 2026-11-11T09:57:52.597633+05:30
Log lines 1-8 of 8

```
==== Running toppings_price ====
Enter toppings as CSV (e.g. cheese,olives): Toppings Price: 70

==== Running pizza_inches ====
Enter pizza size (S/M/L): Size in Inches: 10

==== Running split_bill ====
Enter total and people (e.g. 900/3): Per Person: 300.0
```

Browser capture of the saved execution log. Command output is unedited; long lines wrap.

02 original invalid

Arundhathi K | PES1UG24CS082 | Lab 7

Command: python original/processor.py
Input lines: cheese,olives | M | 900/0
Exit code: 1 | 2026-11-11T09:57:52.651502+05:30
Log lines 1-17 of 17

```
==== Running toppings_price ====
Enter toppings as CSV (e.g. cheese,olives): Toppings Price: 70

==== Running pizza_inches ====
Enter pizza size (S/M/L): Size in Inches: 10

==== Running split_bill ====
Enter total and people (e.g. 900/3): Traceback (most recent call last):
  File "C:\Users\karib\Documents\Codex\2026-11-11\dr-sapna-v-m-pesu-rr\outputs\Lab7\original\processor.py", line
34, in <module>
    main()
  File "C:\Users\karib\Documents\Codex\2026-11-11\dr-sapna-v-m-pesu-rr\outputs\Lab7\original\processor.py", line
30, in main
    result = split_bill(user_input)
             ^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\karib\Documents\Codex\2026-11-11\dr-sapna-v-m-pesu-rr\outputs\Lab7\original\processor.py", line
15, in split_bill
    return round(int(total) / int(people), 2)
                 ~~~~~~~~~~~^~~~~~~~~~~~~~
ZeroDivisionError: division by zero
```

Browser capture of the saved execution log. Command output is unedited; long lines wrap.

Failing fuzz tests - 1 of 3

Same tests run against original.processor; six failures are expected.

03 fuzz failures

Arundhathi K | PES1UG24CS082 | Lab 7

Command: python -m pytest test_processor.py -k no_crash -v --tb=short --color=no
PROCESSOR_MODULE=original.processor
Exit code: 1 | 2026-11-11T09:57:52.697085+05:30
Log lines 1-42 of 109

```
============================= test session starts =============================
platform win32 -- Python 3.11.9, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\karib\Documents\Codex\2026-11-11\dr-
sapna-v-m-pesu-rr\work\test-env\Scripts\python.exe
cachedir: .pytest_cache
hypothesis profile 'default'
rootdir: C:\Users\karib\Documents\Codex\2026-11-11\dr-sapna-v-m-pesu-rr\outputs\Lab7
configfile: pytest.ini
plugins: hypothesis-6.170.15
collecting ... collected 31 items / 25 deselected / 6 selected

test_processor.py::test_toppings_price_no_crash FAILED                   [ 16%]
test_processor.py::test_pizza_inches_no_crash FAILED                     [ 33%]
test_processor.py::test_split_bill_no_crash FAILED                       [ 50%]
test_processor.py::test_toppings_text_no_crash FAILED                    [ 66%]
test_processor.py::test_size_text_no_crash FAILED                        [ 83%]
test_processor.py::test_bill_text_no_crash FAILED                        [100%]

================================== FAILURES ===================================
_____________________________ test_toppings_price_no_crash _____________________________
test_processor.py:28: in test_toppings_price_no_crash
    def test_toppings_price_no_crash(value):
                  ^^^
test_processor.py:29: in test_toppings_price_no_crash
    result = processor.toppings_price(value)
             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
original\processor.py:6: in toppings_price
    for t in toppings_csv.split(','):
             ^^^^^^^^^^^^^^^^^^
E   AttributeError: 'NoneType' object has no attribute 'split'
E   Failing test case: test_toppings_price_no_crash(
E       value=None,
E   )
_____________________________ test_pizza_inches_no_crash _____________________________
test_processor.py:39: in test_pizza_inches_no_crash
    def test_pizza_inches_no_crash(value):
                  ^^^
test_processor.py:43: in test_pizza_inches_no_crash
    assert processor.pizza_inches(value) == expected
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
original\processor.py:11: in pizza_inches
    return SIZES[size.strip().upper()]
                ^^^^^^^^^^
E   AttributeError: 'NoneType' object has no attribute 'strip'
```

Browser capture of the saved execution log. Command output is unedited; long lines wrap.

Failing fuzz tests - 2 of 3

Same tests run against original.processor; six failures are expected.

03 fuzz failures

Arundhathi K | PES1UG24CS082 | Lab 7

Command: python -m pytest test_processor.py -k no_crash -v --tb=short --color=no
PROCESSOR_MODULE=original.processor
Exit code: 1 | 2026-11-11T09:57:52.697085+05:30
Log lines 43-84 of 109

```
E   Failing test case: test_pizza_inches_no_crash(
E       value=None,
E   )
_____________________________ test_split_bill_no_crash _____________________________
test_processor.py:47: in test_split_bill_no_crash
    def test_split_bill_no_crash(value):
                 ^^^
test_processor.py:48: in test_split_bill_no_crash
    result = processor.split_bill(value)
             ^^^^^^^^^^^^^^^^^^^^^^^^^^^
original\processor.py:14: in split_bill
    total, people = text.split('/')
                    ^^^^^^^^^^
E   AttributeError: 'NoneType' object has no attribute 'split'
E   Failing test case: test_split_bill_no_crash(
E       value=None,
E   )
_____________________________ test_toppings_text_no_crash _____________________________
test_processor.py:82: in test_toppings_text_no_crash
    def test_toppings_text_no_crash(value):
                 ^^^
test_processor.py:83: in test_toppings_text_no_crash
    result = processor.toppings_price(value)
             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
original\processor.py:7: in toppings_price
    total += TOPPINGS[t.strip().lower()]
             ^^^^^^^^^^^^^^^^^^^^^^^^^^
E   KeyError: ''
E   Failing test case: test_toppings_text_no_crash(
E       value='',  # or any other generated value
E   )
_____________________________ test_size_text_no_crash _____________________________
test_processor.py:88: in test_size_text_no_crash
    def test_size_text_no_crash(value):
                 ^^^
test_processor.py:89: in test_size_text_no_crash
    assert processor.pizza_inches(value) in (None, 8, 10, 12)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
original\processor.py:11: in pizza_inches
    return SIZES[size.strip().upper()]
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^
E   KeyError: ''
```

Browser capture of the saved execution log. Command output is unedited; long lines wrap.

Failing fuzz tests - 3 of 3

Same tests run against original.processor; six failures are expected.

03 fuzz failures

Arundhathi K | PES1UG24CS082 | Lab 7

Command: python -m pytest test_processor.py -k no_crash -v --tb=short --color=no
PROCESSOR_MODULE=original.processor
Exit code: 1 | 2026-11-11T09:57:52.697085+05:30
Log lines 85-109 of 109

```
E   Failing test case: test_size_text_no_crash(
E       value='',  # or any other generated value
E   )
_____________________________ test_bill_text_no_crash ______________________________
test_processor.py:93: in test_bill_text_no_crash
    def test_bill_text_no_crash(value):
                    ^^^
test_processor.py:94: in test_bill_text_no_crash
    result = processor.split_bill(value)
             ^^^^^^^^^^^^^^^^^^^^^^^^^^^
original\processor.py:14: in split_bill
    total, people = text.split('/')
    ^^^^^^^^^^^^
E   ValueError: not enough values to unpack (expected 2, got 1)
E   Failing test case: test_bill_text_no_crash(
E       value='',  # or any other generated value
E   )
=========================== short test summary info ===========================
FAILED test_processor.py::test_toppings_price_no_crash - AttributeError: 'Non...
FAILED test_processor.py::test_pizza_inches_no_crash - AttributeError: 'NoneT...
FAILED test_processor.py::test_split_bill_no_crash - AttributeError: 'NoneTyp...
FAILED test_processor.py::test_toppings_text_no_crash - KeyError: ''
FAILED test_processor.py::test_size_text_no_crash - KeyError: ''
FAILED test_processor.py::test_bill_text_no_crash - ValueError: not enough va...
====================== 6 failed, 25 deselected in 1.10s =======================
```

Browser capture of the saved execution log. Command output is unedited; long lines wrap.

Bug analysis and fixes

Minimal generated failures guide explicit input validation.

toppings_price

Generated None raises AttributeError; an empty string raises KeyError. Unknown and empty CSV tokens also fail dictionary lookup. The fix checks input type and length, validates every normalized token, then computes the price. It rejects the entire malformed list rather than silently charging for only some toppings.

pizza_inches

Generated None raises AttributeError and an empty string raises KeyError. The fix checks input type and length, normalizes text, and uses a safe lookup. S, M, and L still return 8, 10, and 12 inches.

split_bill

Generated None raises AttributeError and an empty string raises ValueError during unpacking. The original application run with 900/0 separately confirms ZeroDivisionError. The fix validates exactly two bounded unsigned integer fields and requires a positive divisor before converting and dividing.

Regression coverage

Valid examples retain 70 for cheese,olives; 10 for M; and 300.0 for 900/3. Tests also cover 0/1, repeated toppings, case and whitespace, negative values, decimal text, extra separators, huge integers, oversized input, and the 4096/4097-character size boundary. The None sentinel keeps a valid zero bill distinct from an invalid bill.

Reproduction

From Lab7, install requirements-dev.txt, run **python processor.py**, then **python -m pytest test_processor.py -v**. Set PROCESSOR_MODULE to original.processor and add **-k no_crash** to reproduce the six generated failures. Unset the variable before the final corrected run.

Corrected application execution

The valid result is retained; 900/0 receives a validation message.

04 corrected valid

Arundhathi K | PES1UG24CS082 | Lab 7

Command: python processor.py
Input lines: cheese,olives | M | 900/3
Exit code: 0 | 2026-11-11T09:57:54.447349+05:30
Log lines 1-8 of 8

```
==== Running toppings_price ====
Enter toppings as CSV (e.g. cheese,olives): Toppings Price: 70

==== Running pizza_inches ====
Enter pizza size (S/M/L): Size in Inches: 10

==== Running split_bill ====
Enter total and people (e.g. 900/3): Per Person: 300.0
```

Browser capture of the saved execution log. Command output is unedited; long lines wrap.

05 corrected invalid

Arundhathi K | PES1UG24CS082 | Lab 7

Command: python processor.py
Input lines: cheese,olives | M | 900/0
Exit code: 0 | 2026-11-11T09:57:54.515375+05:30
Log lines 1-8 of 8

```
==== Running toppings_price ====
Enter toppings as CSV (e.g. cheese,olives): Toppings Price: 70

==== Running pizza_inches ====
Enter pizza size (S/M/L): Size in Inches: 10

==== Running split_bill ====
Enter total and people (e.g. 900/3): Invalid input: use total/people with people greater than zero.
```

Browser capture of the saved execution log. Command output is unedited; long lines wrap.

Final test execution

All 31 tests pass against the corrected processor.py; process exit code 0.

06 final tests

Arundhathi K | PES1UG24CS082 | Lab 7

Command: python -m pytest test_processor.py -v --tb=short --color=no
Exit code: 0 | 2026-11-11T09:57:54.577125+05:30
Log lines 1-42 of 42

```
============================= test session starts =============================
platform win32 -- Python 3.11.9, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\karib\Documents\Codex\2026-11-11\dr-
sapna-v-m-pesu-rr\work\test-env\Scripts\python.exe
cachedir: .pytest_cache
hypothesis profile 'default'
rootdir: C:\Users\karib\Documents\Codex\2026-11-11\dr-sapna-v-m-pesu-rr\outputs\Lab7
configfile: pytest.ini
plugins: hypothesis-6.170.15
collecting ... collected 31 items

test_processor.py::test_toppings_price_no_crash PASSED                   [  3%]
test_processor.py::test_pizza_inches_no_crash PASSED                     [  6%]
test_processor.py::test_split_bill_no_crash PASSED                       [  9%]
test_processor.py::test_toppings_valid_sum PASSED                        [ 12%]
test_processor.py::test_size_normalization PASSED                        [ 16%]
test_processor.py::test_bill_valid_result PASSED                         [ 19%]
test_processor.py::test_bill_zero_people_rejected PASSED                 [ 22%]
test_processor.py::test_toppings_text_no_crash PASSED                    [ 25%]
test_processor.py::test_size_text_no_crash PASSED                        [ 29%]
test_processor.py::test_bill_text_no_crash PASSED                        [ 32%]
test_processor.py::test_size_input_length_boundary[4096-10] PASSED       [ 35%]
test_processor.py::test_size_input_length_boundary[4097-None] PASSED     [ 38%]
test_processor.py::test_toppings_invalid_tokens[] PASSED                 [ 41%]
test_processor.py::test_toppings_invalid_tokens[,] PASSED                [ 45%]
test_processor.py::test_toppings_invalid_tokens[cheese,] PASSED          [ 48%]
test_processor.py::test_toppings_invalid_tokens[cheese,,onion] PASSED    [ 51%]
test_processor.py::test_toppings_invalid_tokens[tomato] PASSED           [ 54%]
test_processor.py::test_bill_invalid_format[-1/2] PASSED                 [ 58%]
test_processor.py::test_bill_invalid_format[10/-2] PASSED                [ 61%]
test_processor.py::test_bill_invalid_format[1.5/2] PASSED                [ 64%]
test_processor.py::test_bill_invalid_format[10/2/3] PASSED               [ 67%]
test_processor.py::test_bill_invalid_format[10] PASSED                   [ 70%]
test_processor.py::test_bill_invalid_format[/] PASSED                    [ 74%]
test_processor.py::test_bill_invalid_format[+1/2] PASSED                 [ 77%]
test_processor.py::test_bill_invalid_format[nan/2] PASSED                [ 80%]
test_processor.py::test_bill_invalid_format[inf/2] PASSED                [ 83%]
test_processor.py::test_bill_invalid_format[1_000/2] PASSED              [ 87%]
test_processor.py::test_bill_invalid_format[9999999999999/1] PASSED      [ 90%]
test_processor.py::test_oversized_input_rejected[oversized-toppings] PASSED [ 93%]
test_processor.py::test_oversized_input_rejected[oversized-size] PASSED  [ 96%]
test_processor.py::test_oversized_input_rejected[oversized-bill] PASSED  [100%]

============================= 31 passed in 1.24s ==============================
```

Browser capture of the saved execution log. Command output is unedited; long lines wrap.

Source: processor.py

Complete corrected listing | Part 1 of 2

```
"""Pizza order input processing; invalid input returns None."""
import re

TOPPINGS = {"cheese": 40, "olives": 30, "onion": 20, "paneer": 50}
SIZES = {"S": 8, "M": 10, "L": 12}
MAX_INPUT_LENGTH = 4096
BILL_PATTERN = re.compile(r"\s*([0-9]{1,12})\s*/\s*([0-9]{1,12})\s*")


def toppings_price(toppings_csv):
    """Sum a nonempty CSV of known toppings; reject empty/unknown tokens."""
    if not isinstance(toppings_csv, str) or len(toppings_csv) > MAX_INPUT_LENGTH:
        return None
    tokens = [token.strip().lower() for token in toppings_csv.split(",")]
    if any(token not in TOPPINGS for token in tokens):
        return None
    return sum(TOPPINGS[token] for token in tokens)


def pizza_inches(size):
    """Accept S/M/L with case and surrounding whitespace normalization."""
    if not isinstance(size, str) or len(size) > MAX_INPUT_LENGTH:
        return None
    return SIZES.get(size.strip().upper())


def split_bill(text):
    """Split a nonnegative integer total among a positive integer count.

    Each field accepts 1-12 ASCII digits. The bounds prevent huge integer
    conversions and float overflow; rounding follows the original application.
    """
    if not isinstance(text, str) or len(text) > MAX_INPUT_LENGTH:
        return None
    match = BILL_PATTERN.fullmatch(text)
```

Source: processor.py

Complete corrected listing | Part 2 of 2

```
    if match is None:
        return None
    total, people = int(match[1]), int(match[2])
    if people == 0:
        return None
    return round(total / people, 2)


def main():
    operations = [
        ("toppings_price", "Enter toppings as CSV (e.g. cheese,olives): ",
         toppings_price, "Toppings Price:", "use known, nonempty topping names"),
        ("pizza_inches", "Enter pizza size (S/M/L): ",
         pizza_inches, "Size in Inches:", "choose S, M, or L"),
        ("split_bill", "Enter total and people (e.g. 900/3): ",
         split_bill, "Per Person:", "use total/people with people greater than zero"),
    ]
    for name, prompt, function, label, hint in operations:
        print(f"==== Running {name} ====")
        try:
            value = input(prompt)
        except (EOFError, KeyboardInterrupt):
            print("\nInput cancelled.")
            return
        result = function(value)
        if result is None:
            print(f"Invalid input: {hint}.")
        else:
            print(label, result)
        print()


if __name__ == "__main__":
    main()
```

Source: test_processor.py

Complete corrected listing | Part 1 of 3

```
"""Fuzz input validation and preserve valid pizza order behaviour.

Invalid input returns None; valid zero totals remain distinguishable.
PROCESSOR_MODULE=original.processor replays the same tests on the starter.
"""
import importlib
import os
import re

import pytest
from hypothesis import example, given, settings, strategies as st

processor = importlib.import_module(os.getenv("PROCESSOR_MODULE", "processor"))
settings.register_profile("lab7", max_examples=200, deadline=None,
                          derandomize=True, database=None, report_multiple_bugs=False)
settings.load_profile("lab7")

PRICES = {"cheese": 40, "olives": 30, "onion": 20, "paneer": 50}
SIZES = {"S": 8, "M": 10, "L": 12}
arbitrary_input = st.one_of(
    st.text(), st.none(), st.integers(), st.floats(), st.booleans(),
    st.binary(), st.lists(st.integers(), max_size=10),
    st.dictionaries(st.text(max_size=10), st.integers(), max_size=5),
)


@given(arbitrary_input)
def test_toppings_price_no_crash(value):
    result = processor.toppings_price(value)
    expected = None
    if isinstance(value, str) and len(value) <= 4096:
        tokens = [token.strip().lower() for token in value.split(",")]
        if all(token in PRICES for token in tokens):
            expected = sum(PRICES[token] for token in tokens)
    assert result == expected


@given(arbitrary_input)
def test_pizza_inches_no_crash(value):
    expected = None
```

Source: test_processor.py

Complete corrected listing | Part 2 of 3

```
41      if isinstance(value, str) and len(value) <= 4096:
42          expected = SIZES.get(value.strip().upper())
43      assert processor.pizza_inches(value) == expected
44
45
46  @given(arbitrary_input)
47  def test_split_bill_no_crash(value):
48      result = processor.split_bill(value)
49      expected = None
50      if isinstance(value, str) and len(value) <= 4096:
51          match = re.fullmatch(r"\s*([0-9]{1,12})\s*/\s*([0-9]{1,12})\s*", value)
52          if match and int(match[2]) > 0:
53              expected = round(int(match[1]) / int(match[2]), 2)
54      assert result == expected
55
56
57  @given(st.lists(st.sampled_from(list(PRICES)), min_size=1, max_size=30))
58  def test_toppings_valid_sum(names):
59      text = ",".join(" " + name.upper() + " " for name in names)
60      assert processor.toppings_price(text) == sum(PRICES[name] for name in names)
61
62
63  @given(st.sampled_from(list(SIZES)), st.sampled_from(["", " ", "\t", "\n"]))
64  def test_size_normalization(size, whitespace):
65      assert processor.pizza_inches(whitespace + size.lower() + whitespace) == SIZES[size]
66
67
68  @given(st.integers(0, 999_999_999_999), st.integers(1, 999_999_999_999))
69  @example(0, 1)
70  @example(900, 3)
71  @example(100, 3)
72  def test_bill_valid_result(total, people):
73      assert processor.split_bill(f" {total} / {people} ") == round(total / people, 2)
74
75
76  @given(st.integers(min_value=0, max_value=999_999_999_999))
77  def test_bill_zero_people_rejected(total):
78      assert processor.split_bill(f"{total}/0") is None
79
80
```

Source: test_processor.py

Complete corrected listing | Part 3 of 3

```
@given(st.text())
def test_toppings_text_no_crash(value):
    result = processor.toppings_price(value)
    assert result is None or isinstance(result, int) and result >= 0


@given(st.text())
def test_size_text_no_crash(value):
    assert processor.pizza_inches(value) in (None, 8, 10, 12)


@given(st.text())
def test_bill_text_no_crash(value):
    result = processor.split_bill(value)
    assert result is None or isinstance(result, float) and result >= 0


@pytest.mark.parametrize("length,expected", [(4096, 10), (4097, None)])
def test_size_input_length_boundary(length, expected):
    assert processor.pizza_inches(" " * (length - 1) + "M") == expected


@pytest.mark.parametrize("text", ["", ",", "cheese,", "cheese,,onion", "tomato"])
def test_toppings_invalid_tokens(text):
    assert processor.toppings_price(text) is None


@pytest.mark.parametrize("text", ["-1/2", "10/-2", "1.5/2", "10/2/3", "10", "/", "+1/2",
                                  "nan/2", "inf/2", "1_000/2", "9" * 13 + "/1"])
def test_bill_invalid_format(text):
    assert processor.split_bill(text) is None


@pytest.mark.parametrize("function,text", [
    ("toppings_price", "cheese," * 1000),
    ("pizza_inches", "M" * 5000),
    ("split_bill", "9" * 5000 + "/1"),
], ids=["oversized-toppings", "oversized-size", "oversized-bill"])
def test_oversized_input_rejected(function, text):
    assert getattr(processor, function)(text) is None
```